

```

/** FIRST PROBLEM
 * desc: Generate a histogram detailing the frequencies of the random numbers that
fall within a certain range
 *      Bins denoted with 1) center value, 2) frequency. bins are equidistant
(mean +/- x*stdDev) and equal in width (range)
 * notes: The canvas instructions mention rounding into the nearest bin, however
she said that if the widths of the bins dont
 *      touch the other bins, a number generated in between the bins will be
discarded. so presumably that means if the value
 *      is not exactly the center but is within the width, it is counted towards
the bin of the centered value.
 * in: mean
 *      stdDev
 *      numBins (no less than 9)
 *      numSamples
 *      binWidth (no bigger than stdDev)
 * defaults: numBins : 9
 *            numSamples : 20000
 *            binWidth : 1 (catch all numbers)
 * out: a collection of elements, each containing a designation (binNum) and a
frequency
 */

```

```

/* Prototype */
std::vector<std::pair<double, int>> normalDistribution(double mean, double stdDev,
int numBins, int numSamples, double binWidth){}

```

```

/** PSEUDO CODE

```

```

//check for defaults
if numBins < 9
    numBins = 9
if numSamples == -1
    numSamples = 20000
if binWidth equal to -1 or greater than stdDev
    binWidth = stdDev

```

```

/* Generate numbers into a histogram that follow normal distribution (using mean
and stdDev) */

```

```

//declare histogram as a vector of pairs(bin center, frequency) of size numBin
vector<Pair(double binCenter, int freq)> histo(numBin)

```

```

//initialize pairs in histogram

```

```

//start from center (mean) to end

```

```

for from i is numBins/2 and j is 0 to i less than numBins 1 i and j at a time
    histo at i binCenter = mean + j*stdDev
    histo at i freq = 0

```

```

//start from left of center to beginning

```

```

for from i is (numBins/2 -1) and j is -1 to i is less than or equal to 0 1 less i
and j at a time

```

```

    histo at i binCenter = mean + j*stdDev

```

```

    histo at i freq = 0

//declare random number engine
random_device rando{}
mt19937 engine{rando()}
//declare distribution
normal_distribution<double> distr{mean, stdDev}
//declare a temp variables before loop
//to store random number
//the nearest bin to random number
double randNum
double nearBin

/* Helper Function: roundToBin(num) */
double roundToBin(double num){
    //declarations
    double prevBin = histo at 0 binCenter //bin at left
    double nextBin
    //iterate through bins
    for from 1 to numBins
        nextBin = histo at i binCenter //bin at right
        if num less than nextBin
            //check which of the two bins
            if nextBin - num less than equal to num - prevBin
                return nextBin
            else
                return prevBin
        prevBin = nextBin
}

//loop numSample times
for from 0 to numSamples 1 at a time
    //generate a random number
    randNum = distr(engine)

    //if number is within bin width of the nearest bin
    //nearest bin center
    nearBin = roundToBin(randNum)
    //if number is within the upperbound of the lower bin
    if randNum less than or equal to nearBin + half of binWidth
        //look for the bin
        for from 0 to numBins 1 at a time
            if binCenter for the pair at the current index of the histogram is the
nearest bin
                //after the bin has been found
                //increment frequency of that bin
                increment frequency of pair at current index of histo
    //if number is within the lowerbound of the upper bin
    else if randNum greater than or equal to nearBin - half of binWidth
        //look for the bin

```

```

        for from 0 to numBins 1 at a time
            if binCenter for the pair at the current index of the histogram is the
nearest bin
                //after the bin has been found
                //increment frequency of that bin
                increment frequency of pair at current index of histo
            //otherwise, ignore the random number because it does not land within the widths
of the nearest bin
        //end loop

END OF PSUEDO CODE */

```

```

/** SECOND PROBLEM
 * desc: create and return a histogram of random numbers in a uniform distribution
given a range, number of bins, number of samples, and catching range of bins
 * note: distance between bins are (upper-lower)/numBins
 * in: lower
 *         upper
 *         numBins (no less than 21)
 *         numSamples
 *         binWidth (max is the distance between bins)
 * defaults: numBins : 21
 *           numSamples : 20000
 *           binWidth : (upper-lower)/numBins
 * out: a collection of elements, each containing a designation (binNum) and a
frequency
 */

```

```

/* Prototype */
std::vector<std::pair<double, int>> uniformDistribution(double lower, double upper,
int numBins, int numSamples, double binWidth){}

```

```

/** PSEUDO CODE

```

```

//set defaults
if numBins < 21
    numBins = 21
if numSamples == -1
    numSamples = 20000
if binWidth = -1 || binWidth > (upper-lower)/numBins
    binWidth = (upper-lower)/numBins

```

```

//generate empty histogram given the bounds
//declare histogram
vector<pair<double, int>> histo
histo size numBins
//iniitalize the pairs in histo
for from 0 to numBins 1 at a time

```

```

    histo at i binCenter = i * ((upper-lower)/numBins)
    histo at i frequency = 0

//declare random number engine
random_device rando{}
mt19937 engine{rando()}
//declare distribution
uniform_distribution<double> distr{lower, upper}
//declare a temp variables before loop
//to store random number
//the nearest bin to random number
double randNum
double nearBin

//generate a sample numSample times
for from 0 to numSamples 1 at a time
    //generate a random number
    randNum = distr(engine);

    //find the nearest bin center
    nearBin = roundToBin(histo, randNum);

    //if number is within the upperbound of the lower bin
    if randNum less than or equal to nearBin + half of binWidth
        //look for the bin
        for from 0 to numBins 1 at a time
            if binCenter for the pair at the current index of the histogram is the
nearest bin
                //after the bin has been found
                //increment frequency of that bin
                increment frequency of pair at current index of histo

    //if number is within the lowerbound of the upper bin
    else if randNum greater than or equal to nearBin - half of binWidth
        //look for the bin
        for from 0 to numBins 1 at a time
            if binCenter for the pair at the current index of the histogram is the
nearest bin
                //after the bin has been found
                //increment frequency of that bin
                increment frequency of pair at current index of histo

    //otherwise, ignore the random number because it does not land within the widths
of the nearest bin
//end loop

END OF PSUEDO CODE */

```

```

/** THIRD PROBLEM
 * desc: Given a list of students who have not found a group, place them in a group
or create a new one if there are no groups available
 *      All students in the same group must be enrolled in the same section
 *      Groups can have no more than 3 members
 * note: Students have Name, Section
 * in: vector(list) of pairs(student)
 * out: pass by pointer vector(list) of vector(groups) of pairs(student)
 */

```

```

/* Prototype */
void groupAssign(vector<vector<pair<string, int>>> &currList, vector<pair<string,
int>> newList){}

```

```

/** PSUEDO CODE

```

```

//for each student in newList
for from the first element in newList to the last element 1 at a time

    //at each group in currList or until a group is found
    assigned = false
    for from the first element in currList to the last element or a group is found 1
    at a time

        //check that the group is not full
        if the vector at i has less than 3 pairs
            //then check if a person from the group has a matching section
            if pair section is equal to the current pair section from newList
                //add the element to the group in currList
                currList at j push_back(newList at i)
                assigned = true
            //if not then this group is the wrong section; ignore this group
            //if not then the group is full; move on to next group

        //if assigned is still false by the time it reaches the last group, then a new
group must be created
        if not assigned
            //create the group
            vector<pair<int, int>> newGroup(0)
            //add the student to the new group
            newGroup push_back newList at i
            currList push_back newGroup

```

```

END OF PSEUDO CODE*/

```